

The Connected Digital Twin

Group Assignment 1 - Technical Summary

This project implements a connected digital twin of a virtual SUTD lab room. The system establishes a live digital thread from simulated room sensors through MQTT, InfluxDB, a derived twin-state layer, and a Grafana dashboard with persistent temperature alert evidence.

Implementation Enhancements

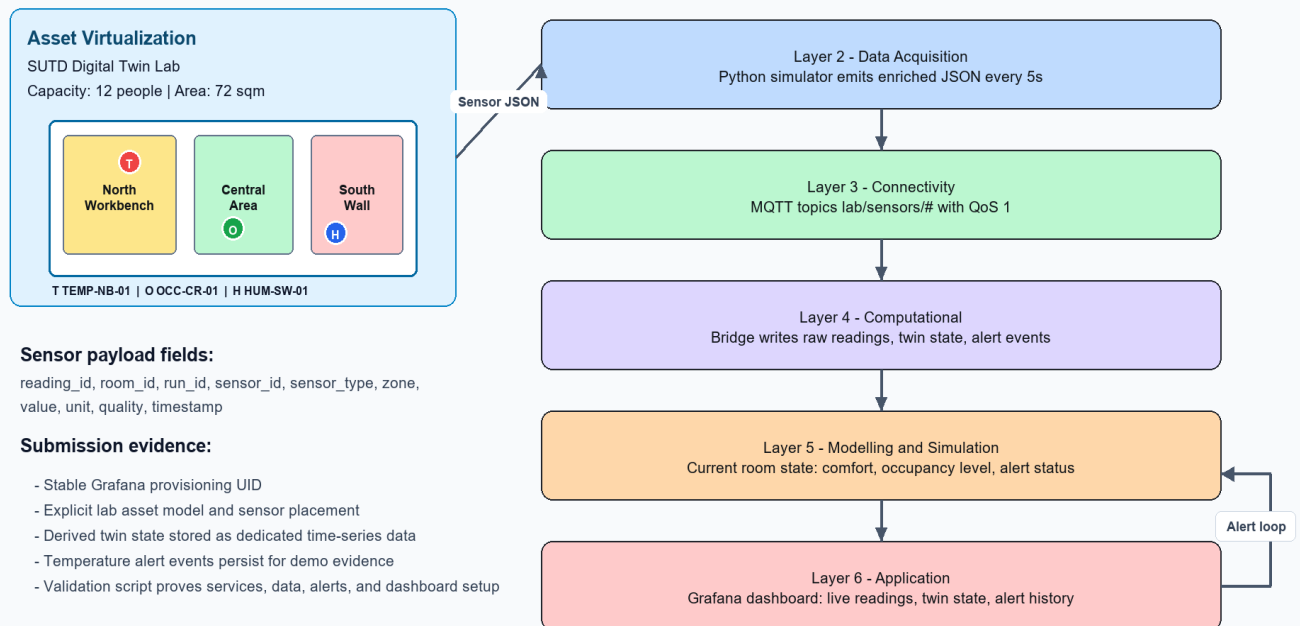
- Asset virtualization is explicit: room metadata, zones, sensor IDs, positions, units, and thresholds are stored in config/lab_asset_model.json.
- Sensor payloads are enriched with run ID, room ID, sensor ID, sensor type, zone, quality, unit, value, and timestamp.
- The bridge writes three evidence streams: raw readings, derived lab_twin_state, and lab_alert_events.
- Grafana provisioning now uses a stable InfluxDB datasource UID so the dashboard survives fresh setup.
- scripts/validate_demo.py performs a forced-alert run and verifies services, dependencies, data, alerts, and dashboard config.

Architecture

The 6-layer architecture maps as follows: physical layer is the virtual SUTD lab; data acquisition is the Python simulator; connectivity is MQTT through Mosquitto; the computational layer is the MQTT bridge and InfluxDB; modelling and simulation is the explicit twin-state derivation; the application layer is Grafana with live state and alert history.

Connected Digital Twin - SUTD Lab Room

Digital thread: sensors -> MQTT -> InfluxDB -> twin state -> Grafana + alert event



Messaging Protocol Choice

MQTT is used because the publish-subscribe model cleanly decouples simulated sensors from downstream consumers. It is lightweight, suitable for frequent IoT sensor messages, supports Quality of Service, and naturally extends to real physical sensors or cloud IoT platforms. Compared with HTTP polling, MQTT reduces unnecessary request overhead and gives the digital twin a live event stream.

Payload Schema

Field	Purpose
reading_id	Unique reading identifier for traceability
room_id / room_name	Links reading to the virtualized asset
run_id	Separates validation/demo runs from older data
sensor_id / sensor_type	Identifies source sensor and measurement type
zone	Connects data to a lab room location
value / unit / quality	Carries measured value with interpretation metadata
timestamp	Preserves the digital thread timing from generation to dashboard

Rubric Mapping

Assignment Criterion	Implemented Evidence
Asset Virtualization	Lab asset model, zones, sensor IDs, sensor positions, thresholds
Synthetic Data Ingestion	Python simulator publishes enriched JSON to MQTT topics
Architecture Mapping	Updated 6-layer diagram with digital thread and alert loop
Dashboard Development	Grafana dashboard shows live streams, current twin state, and alert events
Technical Summary	Protocol rationale, schema, rubric mapping, and validation workflow

Validation Loop

The project includes a repeatable validation command: `python scripts/validate_demo.py`. It checks Docker service status, TCP/HTTP health, Python imports, Grafana provisioning, then runs the simulator with `SIM_FORCE_ALERT=1`. The script queries InfluxDB using the fresh validation run ID and fails if any raw reading, twin-state record, or alert event is missing.

Conclusion

The final prototype goes beyond a basic live chart by making the asset model, sensor metadata, derived room state, alert evidence, and validation loop explicit. This creates a demonstrable connected digital twin rather than only a sensor dashboard.